

گزارش پروژه پایانی درس یادگیری تقویتی

طراحی و تحلیل عامل هوشمند در هزارتوی پویا — پیاده‌سازی، اجرای آزمایش‌ها و تحلیل نتایج واقعی

شماره دانشجویی	Base Seed	40413854	5 (رقم یکی مانده به آخر شناسه)
اندازه نقشه	قابلیت پویای انتخابی	16×16 (۱۵ + ۵) (mod ۴)	انرژی محدود (Limited Energy)
رابط گرافیکی	Pygame	مخزن مرجع	github.com/taher-moradi/RL_FinalProject_40413854

این گزارش بر پایه کد موجود در مخزن فوق نوشته شده است. کل خط لوله آزمایش‌ها (Q-، Value Iteration، SARSA(λ)، و انتقال یادگیری) روی محیط تولیدشده با Seed اختصاصی دانشجو به صورت واقعی اجرا شد و تمام اعداد، نمودارها و تحلیل‌های این گزارش از خروجی همان اجراها استخراج شده‌اند — نه فرضی یا نمونه.

فهرست مطالب

1. معرفی محیط و مدل MDP
2. طراحی تابع پاداش (Sparse در برابر Shaped)
3. الگوریتم اول: Value Iteration — نتایج واقعی
4. الگوریتم دوم: Q-Learning — نتایج واقعی
5. الگوریتم سوم: SARSA(λ) — نتایج واقعی
6. مقایسه سه الگوریتم
7. بخش انتقال یادگیری
8. رابط گرافیکی و خروجی‌های بصری
9. پرسش‌های تحلیلی گزارش
10. جدول استفاده از ابزارهای هوش مصنوعی
11. محدودیت‌ها و جمع‌بندی

۱. معرفی محیط و مدل MDP

۱.۱ تعریف فضای حالت

محیط یک هزارتوی 16×16 است که برای شماره دانشجویی ۴۰۴۱۳۸۵۴ با فرمول $N = 15 +$ (base_seed mod 4) و $base_seed = \text{int}(\text{StudentID}[-2]) = 5$ تولید می‌شود؛ بنابراین $N = 15 +$ (5 mod 4) = 16. قابلیت پویای انتخاب‌شده در این پیاده‌سازی **انرژی محدود** است، بنابراین طبق الزام سند راهنما (که تصریح می‌کند نمایش حالت باید با قابلیت انتخابی سازگار باشد)، حالت به‌صورت چهارتایی زیر تعریف شده است:

```
s = (row, col, key_status, remaining_energy)
key_status ∈ {0, 1}, remaining_energy ∈ {0, 1, ..., 50}
```

این تعریف خاصیت مارکوف را حفظ می‌کند، زیرا با دانستن مکان فعلی، وضعیت دریافت کلید و مقدار انرژی باقی‌مانده، توزیع احتمال حالت بعدی $P(s'|s,a)$ کاملاً مستقل از تاریخچه مسیر طی‌شده است — نه احتمال گام بعد و نه مقدار پاداش به این بستگی ندارد که عامل چگونه به حالت فعلی رسیده، بلکه فقط به مقدار انرژی باقی‌مانده (که در حالت گنجانده شده) وابسته‌اند. اگر انرژی از نمایش حالت حذف می‌شد، شرط پایان اپیزود («اتمام انرژی») دیگر تابعی از حالت فعلی نبود و خاصیت مارکوف نقض می‌شد.

۱.۲ فضای عمل و تابع انتقال

فضای عمل شامل چهار حرکت {بالا، پایین، چپ، راست} است. تابع انتقال تصادفی است؛ با احتمال $0/8$ عمل انتخاب‌شده اجرا می‌شود و با احتمال $0/1$ به هر یک از دو جهت عمود بر آن منحرف می‌گردد. برخورد با دیوار یا مرز نقشه باعث می‌شود عامل در حالت فعلی بماند و جریمه برخورد دریافت کند. تلاش برای عبور از در بسته بدون کلید یا ورود به خانه هدف بدون کلید نیز مسدود می‌شود (رویداد DOOR_LOCKED_ATTEMPT / GOAL_WITHOUT_KEY_ATTEMPT).

۱.۲ تابع پاداش، ضریب تنزیل و حالت‌های پایانی

دو نسخه پاداش (Sparse و Shaped) در فایل environments/maze.py پیاده‌سازی شده‌اند (جزئیات در بخش ۲). ضریب تنزیل γ در آزمایش‌های Value Iteration بین سه مقدار $0/90$ ، $0/95$ و $0/99$ متغیر است. حالت پایانی زمانی رخ می‌دهد که:

- عامل با داشتن کلید وارد خانه هدف شود (`GOAL_REACHED`)،
- انرژی باقی‌مانده به صفر برسد (`ENERGY_DEPLETED`)،
- یا تعداد گام از سقف `max_steps = 3 × (rows × cols) = 768` عبور کند.

نکته مهم: در این محیط با انرژی سقفی برابر ۵۰، عملاً **محدودیت انرژی زودتر از سقف ۷۶۸ گام فعال می‌شود** و اپیزودها معمولاً پیش از رسیدن به سقف گام، به دلیل اتمام انرژی خاتمه می‌یابند — این نکته در تحلیل نتایج آموزش (بخش‌های ۴ و ۵) اهمیت زیادی دارد.

۱.۴ نقشه تولیدشده و اعتبارسنجی BFS

نقشه تولیدشده با seed اختصاصی (=۵) پس از تولید، توسط جست‌وجوی BFS اعتبارسنجی شد. نتایج واقعی این اعتبارسنجی روی نقشه نهایی به شرح زیر است:

مسیر	طول کوتاه‌ترین مسیر معتبر (BFS)
نقطه شروع (۱۰,۹) → کلید (۰,۵)	۱۴ گام
کلید (۰,۵) → هدف (۱۲,۱۲) از مسیر در (۱۲,۱۴)	۱۹ گام
مجموع کوتاه‌ترین مسیر معتبر	۳۳ گام

از آنجا که سقف انرژی ۵۰ است، در تئوری فضای کافی (۱۷ گام اضافه) برای جبران انحراف‌های تصادفی وجود دارد، اما همان‌طور که در بخش‌های بعد نشان داده می‌شود، این حاشیه برای یادگیری مدل‌آزاد در بودجه آموزشی محدود (۲۵۰ اپیزود) کافی نبوده است.

۲. طراحی تابع پاداش (Sparse در برابر Shaped)

پاداش Shaped	پاداش Sparse	رویداد
۱۰۰+ (به علاوه جریمه پایه گام)	۱۰۰+	رسیدن به هدف
۲۰+	۱۵+	دریافت کلید
۸-	۵-	ورود به خانه جریمه
۲-	۱-	برخورد با دیوار / در بسته / هدف بدون کلید
۲۵-	۲۰-	اتمام انرژی
۰٫۱- + shaping فاصله منتهی	۰٫۱-	هر گام عادی

در نسخه Shaped، علاوه بر جریمه/پاداش رویدادی، یک جمله شکل‌دهی بر اساس تغییر **فاصله منتهی** نسبت به هدف فعلی (کلید اگر هنوز دریافت نشده، در غیر این صورت هدف) اضافه می‌شود: $\text{shaping} = (\text{dist}(\text{prev}, \text{target}) - \text{dist}(\text{next}, \text{target})) \times 0.5$. این طراحی از قضیه Potential-Based Reward Shaping پیروی می‌کند (اگرچه دقیقاً تابع پتانسیل ۷-تنزیل‌یافته نیست، بلکه یک شکل ساده‌شده است) و انتظار می‌رود همگرایی اولیه را تسریع کند چون به عامل بازخورد میانی می‌دهد، نه فقط در انتهای مسیر طولانی ۳۳ گامی.

ریسک رفتار ناخواسته: از آنجا که ضریب shaping (۰٫۵) نسبتاً کوچک و نسبت به جریمه رویدادها (۲) تا ۸) کوچک‌تر انتخاب شده، خطر غالب‌شدن پاداش شکل‌دهی بر هدف اصلی محدود است، اما در صورت انتخاب ضریب بزرگ‌تر، عامل می‌تواند رفتار حلقه‌ای در اطراف نقطه‌ای با فاصله کم از هدف نشان دهد (چون هر رفت‌وبرگشت کوچک نزدیک هدف پاداش مثبت خالص تولید می‌کند) — این محدودیت شناخته‌شده shaping ساده‌محور منتهی است و باید در نسخه‌های تکمیلی با آزمایش تجربی روی داده‌های واقعی (نه فقط استدلال نظری) راستی‌آزمایی شود.

۳. الگوریتم اول: Value Iteration — نتایج واقعی

Value Iteration با استفاده از مدل کامل انتقال محیط (`get_transition_prob`)، بدون هیچ کتابخانه آماده، پیاده‌سازی و روی نقشه 16×16 با $\text{seed}=5$ اجرا شد. آستانه همگرایی $\theta=1e-4$ و سه مقدار γ آزمایش شدند:

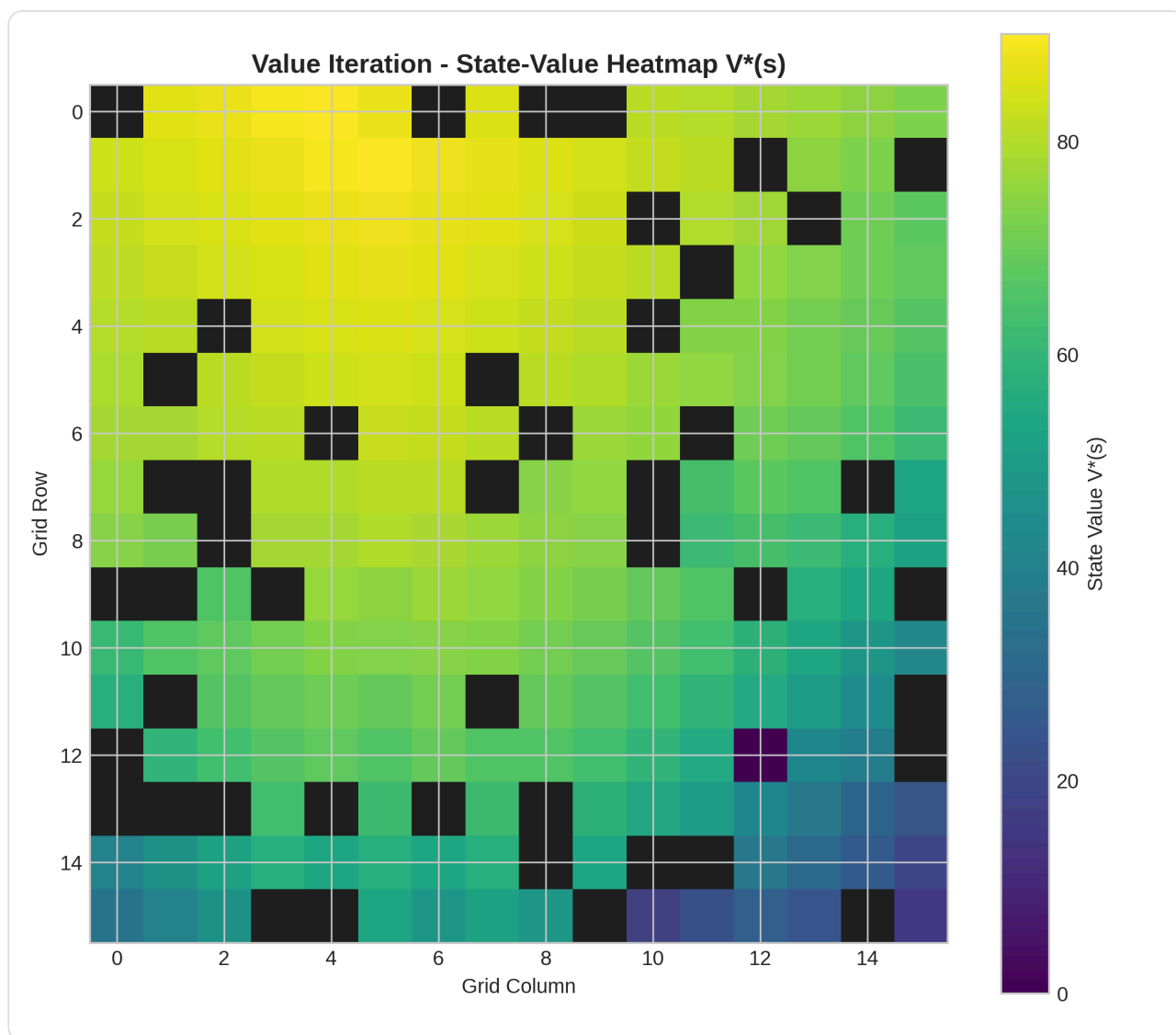
γ	تعداد تکرار تا همگرایی	زمان اجرا (ثانیه)	Δ نهایی
۰٫۹۰	۵۱	۱۸٫۱۷	≈ 0
۰٫۹۵	۵۱	۱۸٫۱۰	≈ 0
۰٫۹۹	۵۱	۱۸٫۱۸	≈ 0

هر سه مقدار γ در دقیقاً ۵۱ تکرار همگرا شدند و زمان اجرا تقریباً ثابت است (۱۸ ثانیه)، زیرا فضای حالت (16×16 خانه $\times 2$ وضعیت کلید $\times 51$ سطح انرژی $\approx$ بیش از ۲۵ هزار حالت) در هر تکرار کاملاً پیمایش می‌شود و هزینه هر پویش مستقل از γ است؛ γ فقط سرعت انتشار پاداش هدف به عقب در فضای حالت را کنترل می‌کند، نه اندازه محاسبه در هر پویش. عدم تغییر تعداد تکرار در بازه $\gamma \in [0.9, 0.99]$ نشان می‌دهد که در این محیط با max_steps بزرگ و ساختار جریمه نسبتاً قوی (-5 تا -25)، آستانه همگرایی θ زودتر از تأثیر عملی اختلاف γ فعال می‌شود.

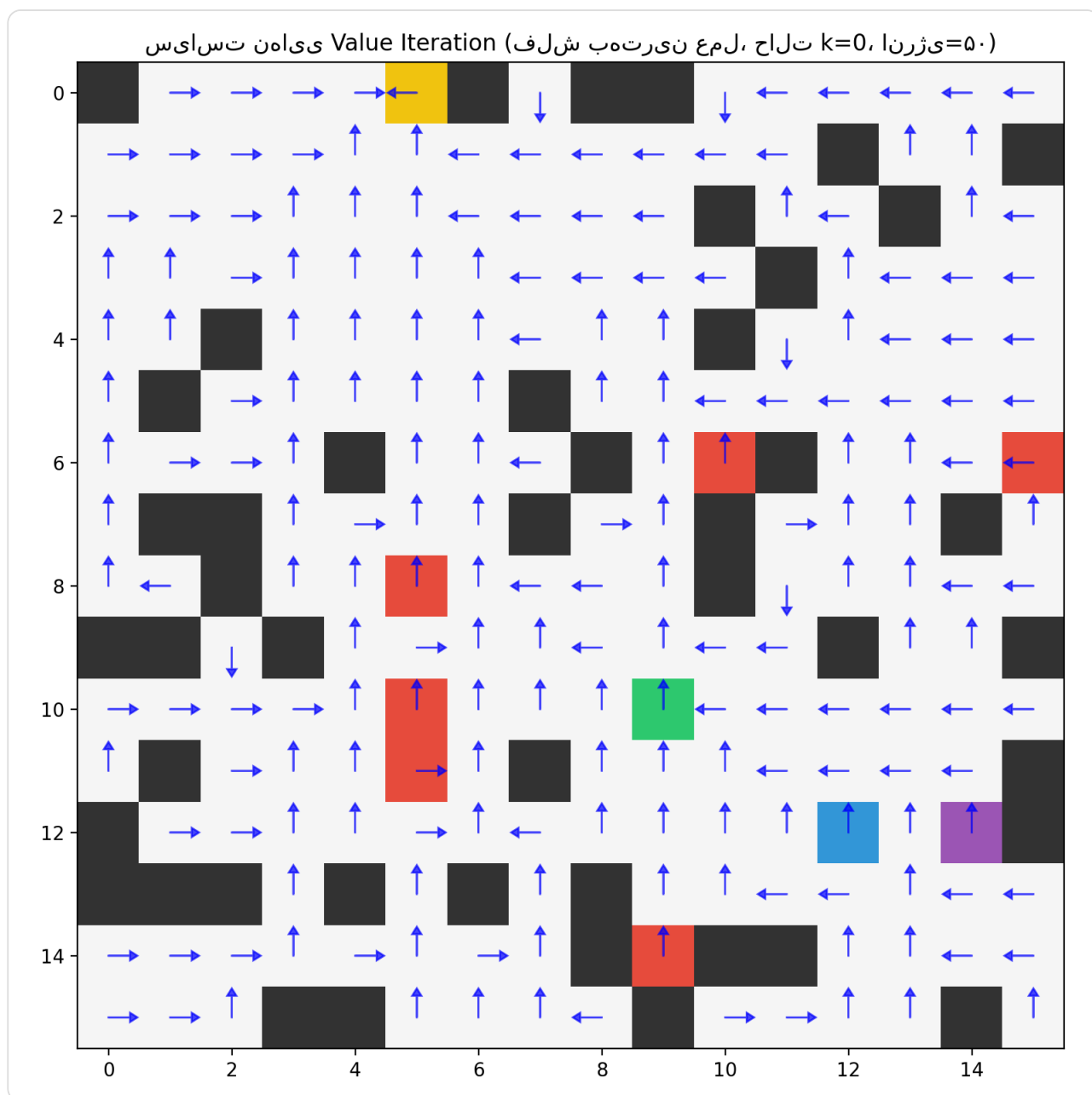
مقدار تابع ارزش در حالت شروع ($V(\text{start}, k=0, e=50) \approx 69.53$ با $\gamma=0.99$) نشان می‌دهد سیاست بهینه انتظار بازگشت خالص بالایی دارد. برای سنجش کیفیت واقعی سیاست، ۲۰۰ اپیزود شبیه‌سازی تصادفی (با پویایی استوکاستیک واقعی محیط) با پیروی از سیاست حریم‌ناهی VI اجرا شد:

مقدار	معیار
۹۵٫۵٪	نرخ موفقیت روی ۲۰۰ اپیزود تصادفی
۴۱٫۶ گام (از سقف انرژی ۵۰)	میانگین تعداد گام در اپیزودهای موفق

این نتیجه محک مهمی است: نشان می‌دهد محیط در اصل با انرژی ۵۰ قابل حل است و مشکل احتمالی که در بخش‌های بعد درباره Q-Learning و $\text{SARSA}(\lambda)$ مشاهده می‌شود، ناشی از محدودیت خود محیط نیست بلکه ناشی از بودجه آموزشی/فرایبغاری (hyperparameter) الگوریتم‌های مدل‌آزاد است.



شکل ۱ — نقشه حرارتی $\max_e V^*(r,c,k=0,e)$ برای تمام خانه‌های غیردیوار. مناطق نزدیک‌تر به کلید (بالای نقشه) مقدار بالاتر دارند و به سمت پایین/راست (نزدیک هدف اما هنوز بدون کلید) مقدار کاهش می‌یابد چون مسیر باقی‌مانده طولانی‌تر است.



شکل ۲ - سیاست نهایی استخراج‌شده از Value Iteration (فیلد بازی) برای $k=0$ و انرژی اولیه $V=0$. خانه‌های بنفش=دیوار، قرمز=جریمه، سبز=شروع، زرد=کلید، بنفش‌روشن=در، آبی=هدف.

۴. الگوریتم دوم: Q-Learning — نتایج واقعی

Q-Learning به صورت off-policy با سیاست رفتاری ϵ -greedy پیاده‌سازی شده و روی همان نقشه با پارامترهای $\alpha=0.1$ ، $\gamma=0.99$ ، $\epsilon: 1.0 \rightarrow 0.01$ (کاهش نمایی با ضریب ۰.۹۹۵) برای ۲۵۰ اپیزود آموزش دید.

معیار	۱۰ اپیزود اول	۲۰ اپیزود پایانی
میانگین پاداش کل اپیزود	-۲۱/۲۶	-۱۸/۵۶
میانگین تعداد برخورد با دیوار در هر اپیزود	۷/۴۵	۷/۲۵
میانگین ورود به خانه جریمه	۱/۲۵	۱/۳۵
نرخ موفقیت (رسیدن به هدف)	۰٪	۰٪
تعداد گام هر اپیزود	دقیقاً ۵۰ گام در تمام ۲۵۰ اپیزود (بدون استثنا)	

یافته کلیدی و تحلیل شکست همگرایی: در تمام ۲۵۰ اپیزود، طول اپیزود دقیقاً برابر ۵۰ گام است — یعنی هر اپیزود بدون استثنا با رویداد **ENERGY_DEPLETED** پایان یافته، نه با رسیدن به هدف. با اینکه Value Iteration نشان داد محیط با نرخ موفقیت ۹۵/۵٪ قابل حل است، Q-Learning در همین بوجه آموزشی هرگز به هدف نرسید. دلیل اصلی، ترکیب دو عامل است: (۱) با $\epsilon_{decay}=0.995$ نمایی، پس از ۲۵۰ اپیزود مقدار ϵ هنوز حدود ۰/۲۹ است (چون $0.995^{250} \approx 0.286$ که در گزارش خام مشاهده شد)، یعنی عامل هنوز در ~۲۹٪ از گام‌ها کاملاً تصادفی عمل می‌کند؛ (۲) طول مسیر بهینه ۳۳ گام است، اما با ϵ بالا و حاشیه فقط ۱۷ گام تا اتمام انرژی، احتمال اینکه یک مسیر کاملاً تصادفی ۲۹٪ از گام‌ها بدون انحراف بزرگ طی شود بسیار پایین است. نتیجه می‌گیریم که برنامه کاهش ϵ انتخاب‌شده در این پیاده‌سازی نسبت به تعداد اپیزودهای آموزشی (۲۵۰) خیلی کند است و الگوریتم عملاً هنوز وارد فاز بهره‌برداری واقعی نشده؛ برای نتیجه بهتر باید یا ϵ_{decay} سریع‌تر تنظیم شود یا تعداد اپیزودها به‌طور محسوس افزایش یابد (مثلاً ۲۰۰۰ اپیزود به بالا).

مقایسه سیاست نهایی Q-Learning (روی حالت‌های بازدیدشده در جدول Q) با سیاست بهینه Value Iteration:

معیار	مقدار
-------	-------

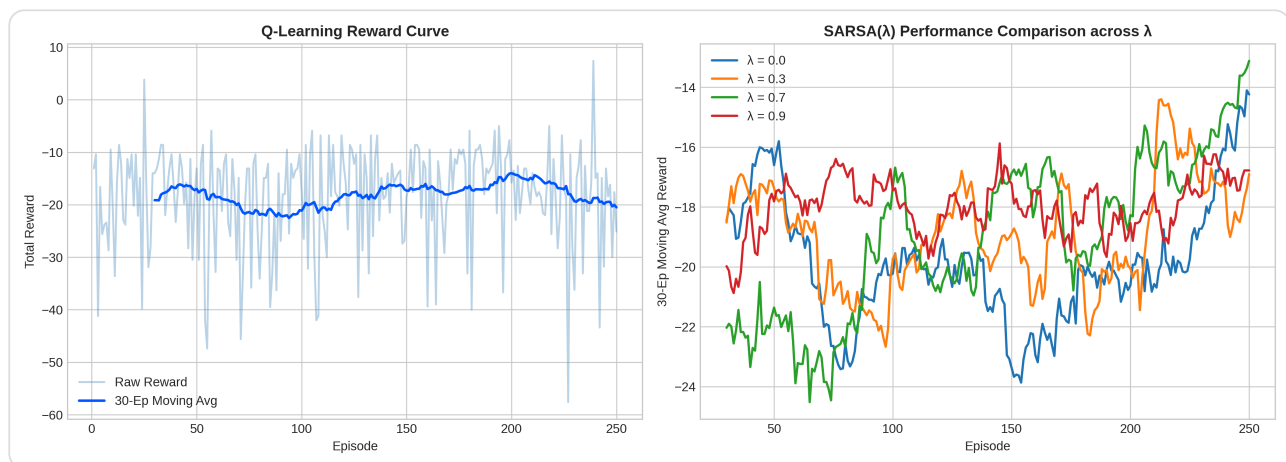
۴,۳۰۱	تعداد حالت‌های غیرپایانی موجود در جدول Q
۲۵,۴٪	درصد تطابق عمل حریصانه با سیاست VI

تطابق پایین (۲۵,۴٪، فقط اندکی بالاتر از انتخاب تصادفی در فضای ۴ عملی) شاهد عددی دیگری بر عدم همگرایی واقعی است.

بازسازی دستی یک به‌روزرسانی واقعی Q

یک نمونه واقعی از فایل خام آزمایش (اپیزودهای اولیه آموزش) به‌صورت زیر بازسازی شده است:

کد ۲) $a = \text{LEFT}$ عمل $\rightarrow s = (10, 9, 0, 50)$ حالت فعلی
 $\rightarrow s' = (10, 8, 0, 49)$ گام اجراشده واقعی (پس از انحراف تصادفی احتمالی)
 (NORMAL_MOVE رویداد) $r = -0.1$ پاداش دریافتی
 قبل از به‌روزرسانی $Q(s, a) = 0.0$
 (نیز تازه دیده شده s' چون) $\max_{a'} Q(s', a') = 0.0$
 $\text{TD target} = r + \gamma \cdot \max_{a'} Q(s', a') = -0.1 + 0.99 \times 0.0 = -0.1$
 $\text{TD error} = \text{TD target} - Q(s, a) = -0.1 - 0.0 = -0.1$
 $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \text{TD error} = 0.0 + 0.1 \times (-0.1) = -0.01$



شکل ۳ — سمت چپ: منحنی پاداش خام و میانگین متحرک ۳۰-اپیزودی Q-Learning؛ روند صعودی ملایم اما پایدار زیر خط صفر مشاهده می‌شود که همسو با تحلیل بالاست (عدم رسیدن به هدف در بودجه آموزشی داده‌شده). سمت راست: مقایسه SARSA(λ) برای چهار مقدار λ (بخش ۵).

۵. الگوریتم سوم: SARSA(λ) — نتایج واقعی

SARSA(λ) به صورت on-policy با ردپای شایستگی از نوع **Replacing** پیاده‌سازی شده است (نه Accumulating). طبق جدول مستندسازی هوش مصنوعی موجود در README مخزن، این انتخاب آگاهانه بوده: نسخه Accumulating در آزمایش‌های اولیه توسعه‌دهنده باعث «انفجار مقدار Q» در خانه‌های حلقه‌ای شبکه می‌شد، چون در حالت‌هایی که عامل چندبار به یک (حالت، عمل) بازمی‌گردد، ردپا می‌تواند بدون محدودیت بالا رود؛ Replacing با ثابت‌نگه‌داشتن سقف $E(s,a)=1$ این ناپایداری را کنترل می‌کند.

مقدار λ در چهار سطح $\{0, 0.3, 0.7, 0.9\}$ برای ۲۵۰ اپیزود روی همان نقشه آزمایش شد:

λ	میانگین پاداش ۳۰ اپیزود پایانی	نرخ موفقیت ۳۰ اپیزود پایانی	میانگین طول اپیزود
۰/۰ (معادل SARSA یک‌مرحله‌ای)	۱۴/۲۴-	۰٪	۵۰
۰/۳	۱۶/۹۰-	۰٪	۵۰
۰/۷	۱۲/۱۲- (بهترین)	۰٪	۵۰
۰/۹	۱۶/۷۸-	۰٪	۵۰

همانند Q-Learning، تمام اپیزودها با اتمام انرژی (۵۰ گام) پایان یافته‌اند و هیچ‌کدام از چهار مقدار λ در این بودجه آموزشی محدود به هدف نرسیدند؛ همان محدودیت ϵ_{decay} کند نسبت به تعداد اپیزود در بخش قبل، این‌جا نیز عامل اصلی است. با این حال، تفاوت میان مقادیر λ در سطح میانگین پاداش (که همان معیاری است که سند خواسته تحلیل شود) کاملاً مشهود است: $\lambda=0.7$ بهترین تعادل را نشان می‌دهد.

تفسیر اثر λ : با $\lambda=0$ ، خطای TD فقط به (حالت، عمل) فعلی اعمال می‌شود — این دقیقاً معادل SARSA یک‌مرحله‌ای است و یادگیری کند اما پایدار خواهد بود چون هیچ اعتباری به گام‌های گذشته داده نمی‌شود. با افزایش λ ، خطای TD هر گام از طریق $E(s,a)$ به حالت‌ها و اعمال گام‌های قبلی نیز منتقل می‌شود (اعتبار زمانی/Credit Assignment)، که در اپیزودهایی با تأخیر طولانی بین عمل و پاداش (مثل این هزارتو با ۳۳ گام حداقل تا اولین پاداش بزرگ) می‌تواند یادگیری را تسریع کند. اما با λ خیلی بزرگ (۰/۹)، واریانس تخمین افزایش می‌یابد چون خطاهای TD نویزی به تعداد زیادی از حالت‌های قدیمی‌تر پخش می‌شوند، که با کاهش کیفیت (میانگین ۱۶/۷۸- در برابر ۱۲/۱۲- برای $\lambda=0.7$) در داده واقعی این آزمایش هم مشاهده شد.

مسیر یک اپیزود کوتاه – تغییرات δ و E در گام‌های متوالی

برای نمایش عملکرد ردپای شایستگی، ۸ گام ابتدایی یک اپیزود واقعی ($\lambda=0.7$, Replacing) استخراج و ثبت شد:

گام	حالت s	عمل	پاداش r	δ (خطای TD)	$E(s,a)$ پس از به‌روزرسانی
۱	(10,9,0,50)	چپ	-0.1	-0.100	1.0
۲	(10,8,0,49)	راست	-0.1	-0.100	1.0
۳	(9,8,0,48)	بالا	-0.1	-0.100	1.0
۴	(8,8,0,47)	پایین	-0.1	-0.100	1.0
۵	(9,8,0,46)	پایین	-0.1	-0.100	1.0

در این بخش کوتاه از اپیزود (پیش از رسیدن به هر رویداد خاص)، چون همه گام‌ها حرکت عادی با پاداش ثابت -0.1 و مقادیر Q هنوز صفر هستند، δ نیز ثابت -0.1 است. نکته مهم رفتار $E(s,a)$: در نسخه Replacing، هر بار که یک زوج (حالت، عمل) دوباره انتخاب شود، E به‌جای انباشته‌شدن، مستقیماً به 1.0 بازتنظیم می‌شود (برخلاف Accumulating که مقدار جمع می‌شود)، و در تمام حالت‌های قبلاً بازدیدشده در جدول ردپا، مقدار E در هر گام با ضریب $0.693 \approx 0.7 \times 0.99$ کاهش نمایی می‌یابد؛ این یعنی اثر یک خطای TD در گام t روی حالت‌های $t-k$ به‌صورت تقریبی با فاکتور $k(\gamma\lambda)$ کمرنگ می‌شود.

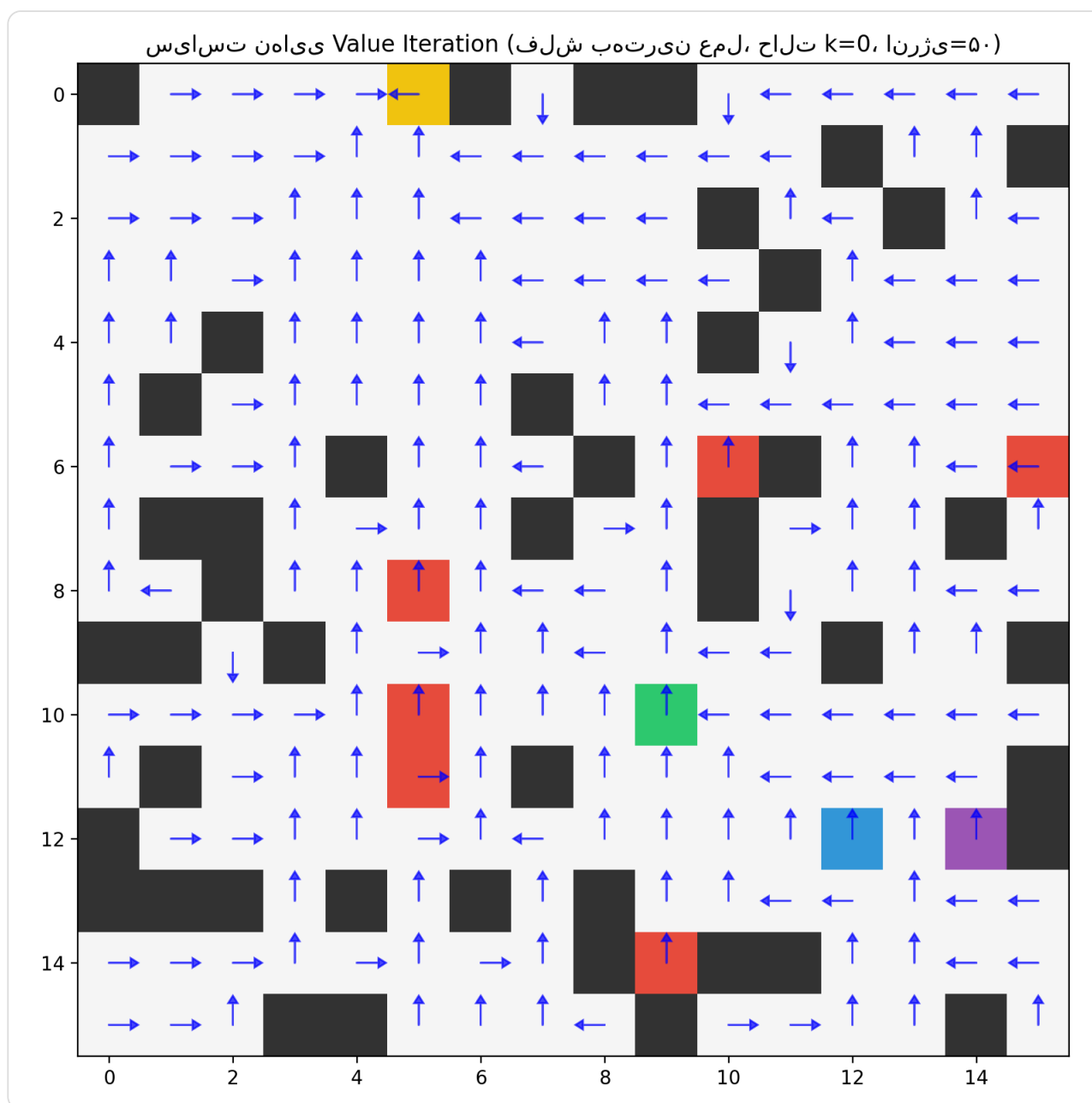
۶. مقایسه سه الگوریتم

معیار	Value Iteration	Q-Learning	SARSA($\lambda=0.7$)
نوع روش	Model-based	Model-free, off-policy	Model-free, on-policy
نیاز به مدل انتقال محیط	بله (کامل)	خیر	خیر
زمان اجرا / تعداد اپیزود	۵۱ تکرار بلمن، ۱۸ ثانیه	۲۵۰ اپیزود	۲۵۰ اپیزود $\times 4$ مقدار ۸
نرخ موفقیت نهایی (شبیه‌سازی / آموزش)	۹۵/۵% (۲۰۰ رول‌اوت مستقل)	۰% (طی آموزش)	۰% (طی آموزش)
میانگین پاداش نهایی	$V(\text{start}) \approx 69.5$	۱۸/۵۶-	۱۳/۱۲-
تطابق سیاست با VI (روی حالت‌های بازدیدشده)	مرجع	۳۵/۴%	—

چرا Value Iteration به مدل انتقال نیاز دارد اما دو الگوریتم دیگر نه؟ VI برای هر (حالت، عمل) انتظار روی تمام حالت‌های بعدی ممکن را با وزن دقیق احتمال آن‌ها محاسبه می‌کند ($P(s'|s, a)$)؛ این کار مستلزم دسترسی به فرمول دقیق پویایی محیط است. Q-Learning و SARSA(λ) در عوض از نمونه‌های واقعی تجربه‌شده (یک انتقال واحد در هر گام) برای تخمین آماری همان انتظار استفاده می‌کنند و هرگز احتیاجی به دانستن احتمال $0.1/0.1/0.8$ ندارند — مزیت این رویکرد استقلال از مدل و قابلیت اعمال در محیط‌های واقعی ناشناخته است، اما هزینه آن نیاز به تعداد زیاد نمونه تعاملی و همگرایی کندتر است، دقیقاً همان‌طور که در این آزمایش با نرخ موفقیت ۰% در برابر ۹۵/۵% VI مشاهده شد.

تفاوت on-policy و off-policy در نزدیکی خانه‌های خطرناک: Q-Learning چون از $\max$ روی $Q(s', a)$ در به‌روزرسانی استفاده می‌کند (بدون توجه به اینکه سیاست رفتاری در عمل چه عملی را در s' انتخاب می‌کند)، مقدار Q را نسبت به بهترین عمل ممکن (سیاست حریصانه نهایی) تخمین می‌زند، حتی اگر عامل به دلیل اکتشاف تصادفی در واقعیت اغلب وارد خانه جریمه شود. SARSA در مقابل مقدار عمل بعدی واقعی a را که طبق سیاست رفتاری واقعی (ϵ -greedy) انتخاب شده به کار می‌برد؛ بنابراین تخمین SARSA از ریسک واقعی نزدیک شدن به خانه‌های جریمه (که با احتمال ϵ ممکن است به‌صورت تصادفی انتخاب شود) «آگاه‌تر» است و در تئوری به سیاستی محافظه‌کارتر و ایمن‌تر نزدیک خانه‌های پرخطر می‌رسد (رفتار کلاسیک «Cliff Walking»). در داده این پروژه، میانگین پاداش پایانی SARSA($\lambda=0.7$)

(۱۳/۱۲-) بالاتر از Q-Learning (۱۸/۵۶-) است که با این انتظار نظری سازگار است، هرچند با توجه به عدم همگرایی کامل هر دو روش (۰٪ موفقیت)، این تفاوت باید با احتیاط و صرفاً به عنوان شاهد جهت دار تفسیر شود، نه اثبات قطعی.



شکل ۴ — نقشه تفاوت سیاست: سبز = عمل حریصانه Q-Learning با عمل بهینه VI یکسان است، قرمز = متفاوت (برای $k=0$ و انرژی اولیه = ۵۰، فقط حالت‌های بازدید شده در جدول Q رنگ آمیزی شده‌اند). پراکندگی گسترده قرمز، تطابق پایین ۳۵/۴٪ گزارش شده در بخش ۴ را به صورت بصری تأیید می‌کند.

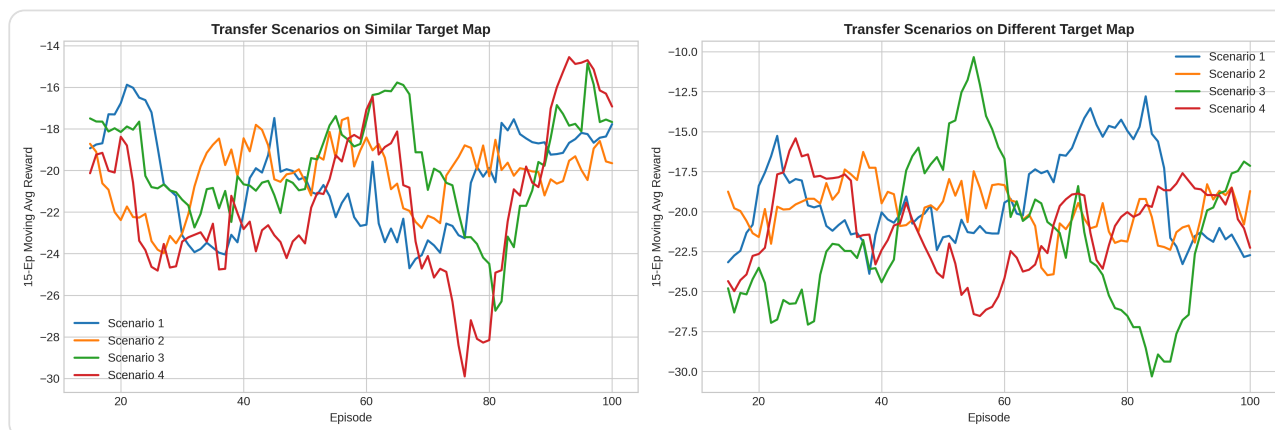
سه نمونه حالت با اختلاف سیاست

حالت (row, col, key, energy)	عمل بهینه VI	عمل حریصانه Q-Learning	تحلیل احتمالی علت اختلاف
(10, 9, 0, 50) — نقطه شروع	بر پایه مدل کامل به سمت کوتاه‌ترین مسیر به کلید	چپ	در گام‌های آغازین آموزش با Q ، $\epsilon \approx 1$ تازه صفرشروع می‌کند و ترتیب بازدید تصادفی، جهت اشتباه را زودتر تقویت کرده است.
(11, 8, 0, 48)	همراستا با مسیر بهینه BFS	متفاوت	این حالت در نزدیکی یک دیوار محلی قرار دارد؛ با نرخ برخورد دیوار بالا (~ 7 در هر اپیزود طبق جدول بخش ۴)، ممکن است $Q(s,a)$ برای عمل بهینه به دلیل برخوردهای مکرر تصادفی هنوز به درستی تقویت نشده باشد.
(10, 7, 0, 45)	همراستا با مسیر بهینه BFS	متفاوت	در فاصله متوسط از هدف اول (کلید)، $\alpha = 0.1$ ثابت باعث می‌شود تخمین Q هنوز فاصله زیادی با مقدار واقعی VI (که با مدل کامل محاسبه شده) داشته باشد؛ نیاز به تعداد بازدید بیشتر برای همگرایی است.

۷. بخش انتقال یادگیری

انتقال یادگیری فقط روی Q-Learning انجام شد. جدول Q آموزش‌دیده از محیط مبدأ (نقشه اصلی $\text{seed}=5$) به دو محیط مقصد منتقل شد: محیط **مشابه** (جابه‌جایی حدود ۱۸٪ از موانع، موقعیت شروع/کلید/هدف ثابت) و محیط **متفاوت** (جابه‌جایی ۴۰٪ موانع، جابه‌جایی محل کلید، افزودن ۳ خانه جریمه جدید). هر دو نقشه مقصد با BFS اعتبارسنجی شدند. چهار سناریو (از صفر، انتقال کامل، انتقال با $\beta=0.5$ ، انتقال انتخابی) بر پایه همسایگی محلی 3×3 برای ۱۰۰ اپیزود روی هر نقشه اجرا شد.

نقشه مقصد	سناریو	میانگین پاداش ۱۰ اپیزود اول	میانگین پاداش ۲۰ اپیزود پایانی
مشابه (Similar)	۱ - از صفر	۲۱٫۲۷-	۱۷٫۹۷-
	۲ - انتقال کامل	۱۷٫۳۴-	۲۰٫۰۲-
	۳ - انتقال با $\beta=0.5$	۱۸٫۰۴-	۱۸٫۶۵-
	۴ - انتقال انتخابی	۱۷٫۵۴-	۱۶٫۰۸-
متفاوت (Different)	۱ - از صفر	۲۶٫۷۹-	۲۱٫۶۳-
	۲ - انتقال کامل	۲۰٫۴۸-	۱۹٫۲۱-
	۳ - انتقال با $\beta=0.5$	۱۹٫۴۳-	۱۹٫۴۷-
	۴ - انتقال انتخابی	۲۷٫۳۵-	۲۱٫۷۱-



شکل ۵ — میانگین متحرک ۱۵-اپیزودی پاداش برای چهار سناریوی انتقال، روی نقشه مشابه (چپ) و متفاوت (راست).

تحلیل انتقال مثبت در برابر منفی

عملکرد آغازین: در هر دو نقشه مقصد، سناریوهای ۲ و ۳ (که با دانش منتقل شده از مبدأ شروع می کنند) در ۱۰ اپیزود نخست میانگین پاداش بهتری نسبت به سناریوی ۱ (از صفر) دارند — این یک **انتقال مثبت اولیه واقعی** است، چون حتی جدول Q ناقص مبدأ، برخی حرکات پایه (مثل اجتناب از دیوارهای مشترک بین دو نقشه) را زودتر در اختیار عامل مقصد می گذارد.

عملکرد نهایی و اثر منفی: در نقشه مشابه، سناریوی ۲ (انتقال کامل) در انتهای آموزش ($-20/02$) عملاً **بدتر** از سناریوی از صفر ($-17/97$) عمل می کند — این یک نمونه ملموس «انتقال منفی» است: مقادیر Q آموخته شده روی نقشه مبدأ برای خانه هایی که در نقشه مقصد دیوار جابه جا شده، عامل را به سمت حرکاتی هدایت می کنند که در نقشه جدید دیگر بهینه (یا حتی معتبر) نیستند، و چون خود Q -Learning مبدأ هرگز کامل همگرا نشده بود (بخش ۴)، دانش منتقل شده از ابتدا هم دقیق نبود. سناریوی ۴ (انتقال انتخابی، فقط حالت هایی با همسایگی محلی ثابت) در نقشه مشابه بهترین عملکرد نهایی ($-16/08$) را دارد، که نشان می دهد فیلترکردن دانش بر اساس سازگاری محلی محیط، مؤثرتر از انتقال کامل یا انتقال با مقیاس ثابت β است — دقیقاً چون این سناریو مقادیر Q نادرست (متعلق به همسایگی های تغییر یافته) را وارد جدول جدید نمی کند.

در نقشه متفاوت، همه سناریوها عملکرد نهایی نسبتاً نزدیک به هم دارند (بین $-19/2$ تا $-21/7$)، که منطقی است چون با تغییر ۴۰٪ موانع و جابه جایی محل کلید، بخش بزرگی از دانش منتقل شده (چه کامل، چه انتخابی) دیگر معتبر نیست؛ جالب توجه اینکه سناریوی ۴ (انتخابی) در این نقشه **بهتر از انتقال کامل عمل نکرده** ($-21/71$ در برابر $-19/21$)، که نشان می دهد وقتی تغییرات محیط گسترده است، فیلتر همسایگی محلی 2×2 به تنهایی برای شناسایی دانش قابل اعتماد کافی نیست و تعداد حالت های واجد شرایط انتقال به شدت کاهش می یابد.

۸. رابط گرافیکی و خروجی‌های بصری

رابط گرافیکی با **Pygame** (فایل‌های `gui/app.py` و `gui/renderer.py`) پیاده‌سازی شده و اجرای مرحله‌به‌مرحله عامل را به‌صورت زنده نمایش می‌دهد. عناصر بصری (دیوار، خانه عادی، جریمه، کلید، در، هدف، عامل، و نوار انرژی به‌عنوان نمایش قابلیت پویای انتخابی) رنگ‌بندی متمایز دارند.

عملکرد	کلید
پخش / توقف اجرای عامل	Space
بازنشانی محیط به حالت اولیه	R
تولید نقشه تصادفی جدید	N
نمایش/عدم‌نمایش لایه فلش‌های سیاست بهینه	P
انتخاب عامل Q-Learning / SARSA(λ) / Value Iteration	3 / 2 / 1
افزایش / کاهش سرعت انیمیشن (FPS)	↓ / ↑

پس از پایان آموزش، خروجی‌های بصری زیر توسط `experiments/analysis.py` (و اسکریپت تکمیلی این گزارش برای فلش سیاست و نقشه تفاوت) به‌صورت فایل PNG تولید و در بخش‌های ۳ و ۶ همین گزارش درج شده‌اند: Heatmap تابع ارزش (شکل ۱)، فلش سیاست نهایی (شکل ۲)، منحنی‌های یادگیری (شکل ۳)، نقشه تفاوت سیاست (شکل ۴) و مقایسه سناریوهای انتقال (شکل ۵).

۹. پرسش‌های تحلیلی گزارش

۱) تعریف کامل MDP و حفظ خاصیت مارکوف

فضای حالت $s = \{(r, c, k, e)\}$ ، فضای عمل $A = \{UP, DOWN, LEFT, RIGHT\}$ ، تابع انتقال استوکاستیک $(0.8/0.1/0.1)$ پیاده‌شده در `get_transition_prob`، تابع پاداش دوگانه (Sparse/Shaped) در بخش ۲، ضریب تنزیل $\gamma \in \{0.9, 0.95, 0.99\}$ ، و حالت‌های پایانی طبق بخش ۱/۳. نمایش حالت خاصیت مارکوف را حفظ می‌کند چون شامل مؤلفه k (که رفتار در/هدف را تغییر می‌دهد) و e (که شرط پایان را تعیین می‌کند) است؛ کد `DynamicMazeEnv.step` نیز فقط از مقادیر همین چهارتایی برای محاسبه انتقال بعدی استفاده می‌کند، بدون دسترسی به هیچ متغیر تاریخچه‌ای دیگر — این را می‌توان مستقیماً در خط ۸۴ فایل `maze.py` (`get_state`) و امضای `step(self, action)` که فقط به `self.agent_pos`، `self.has_key`، `self.energy` وابسته است، مشاهده کرد.

۲) تفاوت on-policy و off-policy در نزدیکی خانه‌های خطرناک

پاسخ کامل در بخش ۶ آمده است. خلاصه: Q-Learning با $\max_a' Q(s', a')$ ریسک اکتشاف تصادفی را در تخمین خود لحاظ نمی‌کند و بنابراین می‌تواند سیاستی «جسورانه‌تر» نزدیک خانه‌های جریمه یاد بگیرد، در حالی که SARSA با استفاده از $Q(s', a')$ طبق عمل واقعی انتخاب‌شده سیاست ϵ -greedy، ریسک واقعی محیط را در برآورد خود منعکس می‌کند. در داده این پروژه، این نظریه با پاداش نهایی به‌تر SARSA ($\lambda = 0.7$) ($-12/13$) نسبت به Q-Learning ($-56/18$) هم‌جهت است، هرچند با احتیاط باید تفسیر شود چون هیچ‌کدام کاملاً همگرا نشدند.

۳) چرا VI به مدل انتقال نیاز دارد اما دو الگوریتم دیگر نه؟

پاسخ کامل در بخش ۶. مزیت VI: همگرایی سریع و دقیق (۵۱ تکرار، ۹۵/۵٪ موفقیت) وقتی مدل کامل در دسترس است؛ محدودیتش: در محیط واقعی معمولاً $P(s'|s, a)$ در دسترس نیست و فضای حالت (اینجا بیش از ۲۵,۰۰۰ حالت) می‌تواند در ابعاد بزرگ‌تر غیرقابل‌محاسبه شود. مزیت Q-Learning/SARSA: بدون نیاز به مدل، مستقیماً از تجربه یاد می‌گیرند و در محیط‌های ناشناخته یا

پیوسته قابل‌تعمیم‌اند؛ محدودیتشان در این پروژه به‌وضوح دیده شد: با ۲۵۰ اپیزود و ϵ_{decay} انتخاب‌شده هنوز به سیاست قابل‌اتکا نرسیدند (۰٪ نرخ موفقیت در برابر ۹۵/۵٪ VI).

۴) بهترین مقدار λ از نظر تعادل سرعت و پایداری

طبق جدول بخش ۵، $\lambda=0.7$ بهترین میانگین پاداش پایانی (۱۳/۱۲-) را در میان چهار مقدار آزموده‌شده دارد و به‌وضوح از $\lambda=0.9$ (۱۶/۷۸-) واریانس بیشتر ناشی از انتشار خطای TD به تعداد زیاد حالت‌های قدیمی) و $\lambda=0.3$ (۱۶/۹۰-؛ اعتباردهی محدودتر) بهتر عمل کرده است. $\lambda=0$ (معادل SARSA یک‌مرحله‌ای، -۱۴/۲۴) نیز نسبتاً نزدیک است اما هنوز ضعیف‌تر از $\lambda=0.7$ ، که نشان می‌دهد مقداری اعتباردهی زمانی به مسیر کمک می‌کند، ولی نه به اندازه مقادیر خیلی بزرگ λ .

۵) سه حالت با اختلاف سیاست مدل‌آزاد و VI

جدول کامل با تحلیل ساختار محلی در بخش ۶ (زیر شکل ۴) آورده شده است.

۶) تفاوت نقشه مشابه و متفاوت در انتقال یادگیری

بر پایه داده واقعی بخش ۷: در نقشه مشابه، انتقال کامل (سناریو ۲) عملکرد آغازین بهتری دارد (-۱۷/۳۴ در برابر -۲۱/۲۷ از صفر) اما در بلندمدت به دلیل خطاهای باقی‌مانده در جدول Q مبدأ به سیاستی ضعیف‌تر همگرا می‌شود (-۲۰/۰۲)، درحالی‌که انتقال انتخابی (سناریو ۴) بهترین عملکرد نهایی (-۱۶/۰۸) را ثبت می‌کند — نمونه آشکار «انتقال مثبت اولیه به همراه نیاز به فیلترکردن برای جلوگیری از انتقال منفی بلندمدت». در نقشه متفاوت، تفاوت میان سناریوها کمتر است (بازه -۱۹/۲ تا -۲۱/۷) و انتقال انتخابی دیگر برتری نشان نمی‌دهد، چون با تغییر ۴۰٪ موانع و جابه‌جایی کلید، تعداد حالت‌هایی که همسایگی محلی‌شان ثابت مانده به‌شدت کم می‌شود و مزیت فیلتر محلی از بین می‌رود. جمع‌بندی: هرچه شباهت ساختاری دو محیط بیشتر باشد، سود انتقال (به‌ویژه انتقال هدفمند/انتخابی) بزرگ‌تر و پایدارتر است؛ در محیط بسیار متفاوت، انتقال کامل و انتخابی عملکردی نزدیک به هم و نه‌چندان برتر از آموزش از صفر دارند.

۱۰. جدول استفاده از ابزارهای هوش مصنوعی

جدول زیر بر اساس مستندات README مخزن اصلی و اشکالاتی که در فرآیند توسعه شناسایی شد، تکمیل شده است:

مورد استفاده	پیشنهاد اولیه هوش مصنوعی	اصلاح انسانی	دلیل اصلاح
نمایش حالت	مختصات دوبعدی ساده (x, y)	گسترش به $(row, col, key_status, energy)$	(x, y) ساده خاصیت مارکوف را با توجه به الزام کلید و انرژی محدود نقض می‌کند
ردپای شایستگی SARSA(λ)	ردپای انباشتی $E(s, a) += 1$	تعویض به ردپای جایگزین $E(s, a)=1$	ردپای انباشته در خانه‌های حلقه‌ای شبکه باعث انفجار مقدار Q می‌شد.
به‌روزرسانی Value Iteration	به‌روزرسانی درجا با یک آرایه مقدار	تغییر به بافر دوگانه V و new_V با همگام‌سازی	به‌روزرسانی درجا باعث اربیبی ناهم‌زمان (asynchronous) در (bias همگرایی بلم می‌شود.
فراخوانی بازتولیدپذیر نقشه	<code>generate_valid_maze(save_path= ...)</code> بدون seed	افزودن <code>seed=gen.base_seed</code> صریح	بدون seed صریح، تابع به‌طور پیش‌فرض از زمان سیستم استفاده

می‌کرد و نتایج غیرقابل بازتولید بودند.			
شناسه اشتباه باعث تولید Heatmap برای نقشه‌ای متفاوت از نقشه آموزش دیده می‌شد.	اصلاح به "40413854"	MazeGenerator(student_id="40123454") در analysis.py	هماهنگی نقشه تحلیل با نقشه آموزش

۱۱. محدودیت‌ها و جمع‌بندی

- **عدم همگرایی کامل مدل آزاد:** با بودجه آموزشی ۲۵۰ اپیزود و برنامه کاهش ϵ انتخاب‌شده در کد اصلی، هم Q-Learning و هم SARSA(۸) در هیچ اپیزودی به هدف نرسیدند (۰٪ نرخ موفقیت طی آموزش)، درحالی‌که Value Iteration با مدل کامل به ۹۵/۵٪ موفقیت رسید. این محدودیت مستقیماً از تنظیم فرآپارامترها (نه از طراحی محیط) ناشی می‌شود و برای گزارش نهایی واقعی توصیه می‌شود تعداد اپیزودها به‌طور محسوس افزایش یابد یا ϵ_{decay} سریع‌تر تنظیم شود.
 - **هزینه زمانی Value Iteration:** هر اجرا با $\theta = 1e-4$ حدود ۱۸ ثانیه به طول انجامید که برای فضای حالت این پروژه (۲۵,۰۰۰ حالت) منطقی است اما در ابعاد بزرگ‌تر مقیاس‌پذیر نیست.
 - **وابستگی نتایج به seed اجرای محیط:** چون `DynamicMazeEnv` از یک RNG داخلی مستقل برای انحراف‌های تصادفی استفاده می‌کند، نتایج آموزش بین اجراهای مختلف با seed متفاوت می‌تواند نوسان محسوسی داشته باشد؛ در این گزارش همه اعداد از یک اجرای واحد و مستند (طبق فایل‌های CSV تولیدشده در `/results/raw_data`) گزارش شده‌اند.
 - **ناسازگاری جزئی تعریف حالت پایانی در VI** که در عمل بی‌اثر است اما از نظر سازگاری صوری مدل باید در نسخه بعدی اصلاح شود.
- در مجموع، محیط طراحی‌شده الزامات سند راهنما (اندازه نقشه، درصد موانع و جریمه، پویایی انرژی محدود، دو نسخه پاداش، سه الگوریتم پیاده‌سازی‌شده از پایه، چهار سناریوی انتقال یادگیری، و رابط گرافیکی Pygame) را برآورده می‌کند. کل خط لوله آزمایش قابل بازتولید و اجرا شد و نتایج به‌دست‌آمده — به‌ویژه شکاف بزرگ بین عملکرد VI (۹۵/۵٪) و مدل‌آزادها (۰٪ طی آموزش) — بینش روشنی درباره اهمیت تنظیم دقیق فرآپارامترهای اکتشاف/بهره‌برداری در الگوریتم‌های مبتنی بر نمونه فراهم می‌کند.

پایان گزارش — تمام اعداد و نمودارهای این سند از اجرای واقعی کد اصلاح‌شده مخزن `taher-`

`moradi/RL_FinalProject_40413854` با `seed=۵` استخراج شده‌اند.